- To create something from a file

  kubectl create -f <file path> [it'll create if resource doesn't exist]

  kubectl apply -f <file path> [create if doesn't exist; update if exists]

- To edit any resource live

  kubectl edit <resource type> <resource name>

  kubectl edit pod my-pod

- To see all the details of a resource along with events

  kubectl describe <resource type> <resource name>

  kubectl describe pod my-pod

- To see *live resource configuration* in *yaml* format

  kubectl get <resource type> <resource name> -o yaml

  kubectl get pod my-pod -o yaml

- Too see all the things of the available resources

  kubectl get <resource type> <resource name> -o wide

- To get a yaml configuration file of any resource to start with,

  --dry-run=client -o yaml append this in the imperative commands.

  kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

- Pod
  - To create a pod ("v1")

    kubectl run <pod name> --image=<image name>

    kubectl run my-pod --image=nginx
  - Whatever ports you write inside containers (inside pods) are not exposed, that is just used for user's docs or to search pods a/c to ports. The traffic reaches to a specific port of a container with Service only.
  - If a pod contains multiple containers,

    each container: *same IP, different ports* [because IP is assigned to Pod]

- ReplicaSet
  - ReplicaSet *("apps/v1")* and ReplicationController *("v1")* cannot be created with imperative commands.
  - ReplicaSet contains *replicaset.spec.selector* field, which ReplicationController doesn't have.
  - *Selector* contains *matchLabels* or *matchExpressions*.

- *template* (pod) should contain same or more labels present in *selector*. Selector's labels work with AND rule, not OR. All the labels must be present in the pods to be selected.
- To scale replica set:

  kubectl scale --replicas=<replica count**> replicaset <**replicaset name**>**

  kubectl scale --replicas=4 replicaset my-rs

- Deployment
  - Deployment can be created with Imperative way because Kubernetes auto-fill the required complex fields like template, labels, selectors.

    kubectl create deployment <deployment name**> --image=<**image name**> --replicas=<**replica count**>**
  - It provide *pull, upgrade, rolling update (or) recreate, rollback (undo), pause (rollout pause), resume (rollout resume)* over ReplicaSet.
  - It create new versions, and *scales down* old ReplicaSet gradually and *scales up* new ReplicaSet gradually.
  - *maxUnavailable* (maximum allowed unavailable pods. Percentage or count), *maxSurge* (maximum allowed available pods. Percentage or count)

    [scale up of new pods & scale down of old pods happens w.r.t. *maxUnavailable* & *maxSurge*]
  - To see *rollout status*,

    kubectl rollout status deployment <deployment name**>**

    To see *rollout history*,

    kubectl rollout history deployment <deployment name**>**

    [for this --record has to be provided; but its deprecated]

    To *undo* the deployment

    kubectl rollout undo deployment <deployment name**>**
  - Deployment = ReplicaSet + [rollout, rollback, versioning(revisions), rollout strategy (Recreate, RollingUpdate), Pause, Resume]
- Deployment Strategy
  - RollingUpdate: gradually remove old pods and create new pods.
  - Recreate: remove all old pods and create new pods.
- Service

- Required to communicate with the pods.

  [Pod's IP can be used; but shouldn't because Pods might get re-created]
- External to Pod: NodePort

  Pod to Pod: ClusterIP (NodePort can also be used; but not preferred)

  Load Balancing: LoadBalancer [works in cloud based clusters]
- type:NodePort

  ports: [{ nodePort: x, port: y, targetPort: z }, { .. }, ..]

  port means service's port

  in here, whatever port you assign for *service*, it doesn't matter because it is there only to connect nodePort to targetPort (pod's port).
- type: ClusterIP

  ports: [{ port: x, targetPort: y }, { .. }, ..]
- For inter-pod communication

  <service name>:<service port>

  service -> name, selector; pod: labels; <-- these are important
- *selector* doesn't contain *matchLabels, matchExpressions*.

  Direct *key:value* pairs (labels) to select the target pods.
- Service's redirection to Pod is cluster level. So service's and pod's host nodes might be different.

- **kubeadm** tool can be used to create a cluster and configure multiple nodes (control plane & worker nodes). it generates the required certificates and all.

  https://github.com/kodekloudhub/certified-kubernetes-administrator-course/tree/master/kubeadm-clusters/generic
- **cgroup** (Control Group)
  - Used to enforce limits by Kernel.
  - **mount | grep cgroup** : to see the mounted path (/sys/fs/cgroup)

    cgroup2 for modern systems; cgroup for older systems;
  - Files & Directories inside *cgroup* are handled by ?
    - all files : *Kernel*
    - directories of format *cgroup.*, cpu.*, memory.*, io.** : *Kernel*
    - directories of format *.slice, .scope, .service, .mount* : ***systemd***
    - other directories (*docker, kubepods ..etc*): respective owners

- *system.slice* contains *containerd.service*, it means container runtime (containerd) is managed by *systemd* itself.
  In case of separately handled, both will not get to know each other's usage and request for more resources; at the end kernel will take action and stop some resources which might be important.
- Owners writes into the respective files, Kernel enforces limits.

- Internals of *kubectl apply*
  - 3 configs are there:
    config file (static yaml configuration file used to create a resource)
    live config file (configuration file of the running resource; *kubectl get <resource type> <resource name> -o yaml*)
    last applied config (latest config applied in *declarative* way) [its in JSON format]
  - Running resource updated with:
    - imperative command (or) *kubectl edit* command: live config is updated, last applied config will remain same
    - Declarative way (*kubectl apply*) : live applied config will be updated, config file will be updated of-course.
  - Configs updation:

    If a field is there in *live config* (ex: an extra level was added using *kubectl edit* command) but not in both *config file* and *last applied config* then field will remain as it is.

    If a field is there in either *config file* or *last applied config* or *both* then that field will be updated (doesn't matter if that field is there in *live config* or not)

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

- namespace
  - Namespace defines a logical boundary where resources can be present.

- Same named resources cannot stay inside a single namespace; but can be present in different namespaces.
- Ex: dev, prod, ..etc namespaces can be created.
- To query/update resources in a specific namespace, use
  -n <namespace name> (or) **--namespace <namespace name>**
  kubectl get pods -n dev-namespace
- In the yaml, *metadata.namespace* is the required field to set the namespace.
- Clustered configured with *kubeadm*, contains *kube-system* namespace which contains control plane resources.
- To access a service of another namespace
  <service name>.<target namespace name>.svc.cluster.local
  db-service.dev-namespace.svc.cluster.local

- ResourceQuota
  - Used to limit resources for ***namespace*** level.
  - ```yaml
    spec:
      hard:
        pods: "10"
        requests.cpu: "4"
        requests.memory: 5Gi
        limits.cpu: "10"
        limits.memory: 10Gi
    ```

- Scheduling
  - *Scheduler* updates *pod.spec.nodeName* field of Pod.
    Then request API server to bind Pod to the specific node.
    Binding request contains payload, where *kind: bind*
    ```yaml
    apiVersion: v1
    kind: Binding
    metadata:
      name: test-binding
    target:
      apiVersion: v1
      kind: Node
      name: node01
    ```
    [not a Kubernetes object; just payload of request]
  - curl <request uri../binding> --request POST --
    data '{"apiVersion":"v1", ...}'
    (bind config in JSON format)
    ```

- In case of manual scheduling, mention *pod.spec.nodeName* field and it'll create pod in required node.

- To query resources with specific labels,

  --selector key1=val1,key2=val2 [append this]

  kubectl get all --selector app=nginx,key2=val2

- *metadata.annotations* used for giving extra info like build versin info, adding tool details ..etc without impacting anything.

- Taints & Tolerations
  - Taint is applied on Node; Toleration is applied on Pod.
  - Taint prevent pod to be scheduled to the tainted node;
    If Pod is tolerated against that specific taint then only it can be scheduled to that tainted node.
  - To taint node (*imperative method only*; ~~no declarative method~~)

    kubectl taint node <node name> <key>=<value>:<taint effect>

    kubectl taint node node01 env=prod:NoSchedule
  - To remove taint, just append **-** at the end.

    kubectl taint node <node name> <key>=<value>:<taint effect>**-**

    kubectl taint node node01 env=prod:NoSchedule- (most specific)

    kubectl taint node node01 env=prod- (less specific)

    kubectl taint node node01 env- (more less specific)
  - One node can have more than one same key-value pair taints; but *taint effect* should be different.

    env=prod:NoSchedule

    env=prod:NoExecute

    env=prod:PreferNoSchedule
  - To tolerate the pod (*declarative method only*; ~~no imperative method~~)

    update the field *pod.spec.tolerations* [it contains a list]

    ```
    spec:
      tolerations:
      - key: "env"
        operator: "Equal"
        value: "prod"
        effect: "NoSchedule"
    ```

    operators: *Equal, Exists, Gt, Lt*

    double quote is mandatory here.
  - Taint effects:

- NoSchedule: all current pods will present; non-tolerated pod will not be scheduled.
- PreferNoSchedule: all current pods will present; non-tolerated pod might be scheduled in case of no other option.
- NoExecute: all current non-tolerated pod will be removed; non-tolerated pod will not be scheduled.

- nodeSelector
  - *pod.spec.nodeSelector* contains the labels (key:value pairs)

    all the labels matches with the node => node is scheduled to this pod.

    ```
    spec:
      nodeSelector:
        env: prod
    ```

- nodeAffinity
  - *pod.spec.affinity.nodeAffinity* has to be updated.

    nodeAffinity support complex queries like DoesNotExist, Exists, Gt, In, Lt, NotIn

    which was not possible in *nodeSelector*
  - nodeAffinity is of 2 types:
    - *requiredDuringSchedulingIgnoredDuringExecution* [HARD rule]
    - *preferredDuringSchedulingIgnoredDuringExecution* [SOT rule (Scored basis)]

    ```
    nodeAffinity
    ├── preferredDuringSchedulingIgnoredDuringExecution []
    │   ├── preference
    │   │   ├── matchExpressions []
    │   │   └── matchFields []
    │   └── weight
    └── requiredDuringSchedulingIgnoredDuringExecution
        └── nodeSelectorTerms []
            ├── matchExpressions []
            └── matchFields []
    ```

  - [~~its not *match**Labels***~~, its *match**Fields*** (not limited to labels, can query any field like *metadata.name*)]
  - *matchExpressions*: query1 **AND** query2 **AND** ...

    matchFields: query1 **AND** query2 **AND** ...

```yaml
matchExpressions:
- key: env
  operator: In
  values:
  - prod
- key: disk
  operator: In
  values:
  - ssd
```

```yaml
matchFields:
- key: metadata.name
  operator: In
  values:
  - node01
  - node02
- key: metadata.resourceVersion
  operator: In
  values:
  - "353007"
  - "353008"
```

- *requiredDuringSchedulingIgnoredDuringExecution*

```
requiredDuringSchedulingIgnoredDuringExecution
└── nodeSelectorTerms []   (OR)
    ├── Term 1
    │   ├── matchExpressions []  (AND)
    │   └── matchFields []       (AND)
    │       ⇒ Term 1 = (matchExpressions AND matchFields)
    │
    ├── Term 2
    │   ├── matchExpressions []  (AND)
    │   └── matchFields []       (AND)
    │       ⇒ Term 2 = (matchExpressions AND matchFields)
    │
```

  - nodeSelectorTerms [] = term1 **OR** term2 **OR** ...

    each term = (*matchExpressions* **AND** *matchFields*)
  - Node should satisfy any one *term* (means all queries of *matchExpressions* **AND** *matchFields* of that *term*)
- *preferredDuringSchedulingIgnoredDuringExecution*

```
preferredDuringSchedulingIgnoredDuringExecution []   (scored list)
├── Item 1
│   ├── preference
│   │   ├── matchExpressions []  (AND)
│   │   └── matchFields []       (AND)
│   │       ⇒ match = (matchExpressions AND matchFields)
│   └── weight
│
├── Item 2
│   ├── preference
│   │   ├── matchExpressions []  (AND)
│   │   └── matchFields []       (AND)
│   │       ⇒ match = (matchExpressions AND matchFields)
│   └── weight
│
```

```
[
  { preference: (matchExpressions AND matchFields), weight },
  { ... },
  ...
]
```

- Each items preference:

  weight = *'provided weight'* if match else *'0'* *(zero)*

  total score of node = sum of each item's weight

- requiredDur....Execution: at least one should match

  preferredDur...Execution: more matches, more score

- Resource Requirements
  - Update field: *pod.spec.containers[*].resources*

    requests -> required resources, else node will not be scheduled [scheduling phase].

    limits -> maximum utilization of resources in the node [running phase]

    ```
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"
    ```

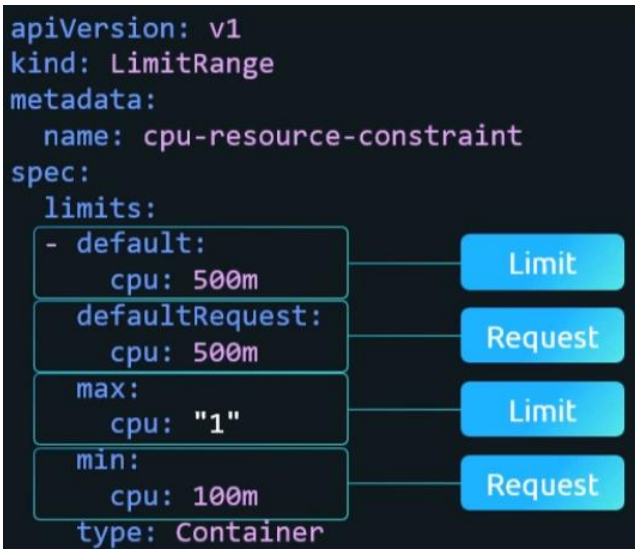
  - cpu smallest unit: **1m** (1 mili CPU), **1** means one full CPU.

    memory smallest unit: **1** (1 byte), use **Gi, Mi, Ki** ~~(don't use G, M, K)~~
  - CPU maximum limit reached: throttled (slow down)

    Memory maximum reached: container is killed
  - Best scenario: Request, No Limit
- LimitRange
  - Inject default *request* and *limit* if not configured inside pods.
  - Multiple LimitRange can be configured in a namespace, but they should not conflict.

    ~~Ex: one says max cpu: 500m, another says min cpu: 600m~~

    ~~in this case, pod will not be created because of conflicting values.~~

- In real life: one LimitRange per namespace is maintained.
- 

  limits.default : limit

  limits.defaultRequest: request

  limits.max: request or limit should not more than it.

  limits.min: request or limit should not less than it.

- ResourceQuota is for namespace level.

  LimitRange is for Pod level.

- DaemonSet

  same as ReplicaSet.

  kind: DaemonSet

  - It make sure one copy of the Pod runs on each node.

- Static Pods

  - Kubelet on each node watches a directory, it Pod's yaml config file is present then it creates a pod.

    It doesn't involve API Server.

  - **systemctl cat kubelet** [see the *kubelet.service* file]

    get the *config.yaml* file path and open that file.

    One key will be there ***staticPodPath***, that is the path to keep the Pod's yaml configs.

  - Kube API Server knows about all the static pods, but you cannot update using *kubectl edit* command.

    The *yaml* config file needs to be changed to update the static pod.

- Pod creation internals

- kubectl apply -f my-pod.yaml [pod creation request triggered]
- API server validates, & creates Pod object. Update etcd about *PodSpec*.
- Scheduler sees an unscheduler pod, assign a node.
- *Kubelet* has a watch connection with API Server. Changes are streamed to kubelet from API server through that (its not webhook; kind of streaming type)

```
GET /api/v1/namespaces/test/pods?watch=1&sendInitialE
---
200 OK
Transfer-Encoding: chunked
Content-Type: application/json

{
  "type": "ADDED",
  "object": {"kind": "Pod", "apiVersion": "v1", "meta
}
{
  "type": "ADDED",
  "object": {"kind": "Pod", "apiVersion": "v1", "meta
}
{
  "type": "BOOKMARK",
```

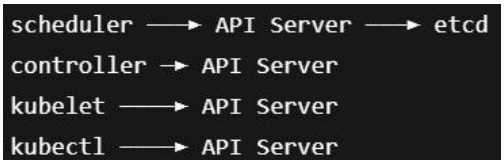
- Kubelet sees PodSpec, pull it, command Container Runtime.
- Container Runtime pull the required image, create the containers, assigns Container IDs.
- Kubelet maps *Pod ID -> Container IDs* .
  monitors containers (running/crashed/restarted)
- Kublet updates API Server about Pod's status (send details about containers like container IDs, pod status, ..etc)
- API Server stores details in etcd.
  [each few seconds, Kubelet updates API Server about the pod status, container details ..etc]
- Container runtime knows about container only (container ID and all)
  Kubelet knows about PodSpec (map it to container IDs received from container runtime)
  API Server know keeps info that got from Kubelet only.
- Each time we execute *kubectl edit* or *kubectl apply -f*, API Server sends PodSpec and the process continues.

- In case of static pods, Kubelet sends a few extra fields

  metadata.annotations.”kubernetes.io/config.source”: file

  *metadata.annotations.”kubernetes.io/config.**mirror**”: <hash>*

- Resources communication

  - All resources talks to API Server, API server talks to *etcd*.

    All the communications happens via HTTP calls.

  - So, even if the resources are running in the node directly

    (or)

    the resources are running inside pods, it doesn’t matter.

    Node to Pod will have some port mapping (in case of *kubeadm*, port mappings are same i.e. 6443:6443 ..etc)

  - The pods that is running resources, must be static. Because API Server shouldn’t create or delete these.

  - ***Kubelet*** doesn’t run inside any Pod. It is a ***systemd*** service.

- Multiple Schedulers

  - 3 files are involved:

    kube-scheduler.yaml [kind: Pod , static pod that runs scheduler]

    scheduler.conf [config file containing *cluster, user, context* details]

    scheduler-config.yaml [kind: KubeSchedulerConfiguration; optional]

  - Either you mention the things in *pod.spec.containers[0].command* field without mentioning *scheduler-config.yaml* file [minimal configs like **--leader-elect=true**]

    (or)

    mention the *scheduler-config.yaml* file for proper configuration.

    --config=/etc/kubernetes/scheduler-config.yaml [you can give any name]

```
spec:
  containers:
    - name: kube-scheduler
      image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
      command:
        - kube-scheduler
        - --address=127.0.0.1
        - --kubeconfig=/etc/kubernetes/scheduler.conf
        - --config=/etc/kubernetes/my-scheduler-config.yaml
```

- To use this scheduler to schedule your pod,

  pod.spec.schedulerName field has to be updated.

  ```
  schedulerName: my-custom-scheduler
  ```
  [pod.spec.schedulerName]

- Scheduler Profiles
  - Scheduler is a process (one pod)

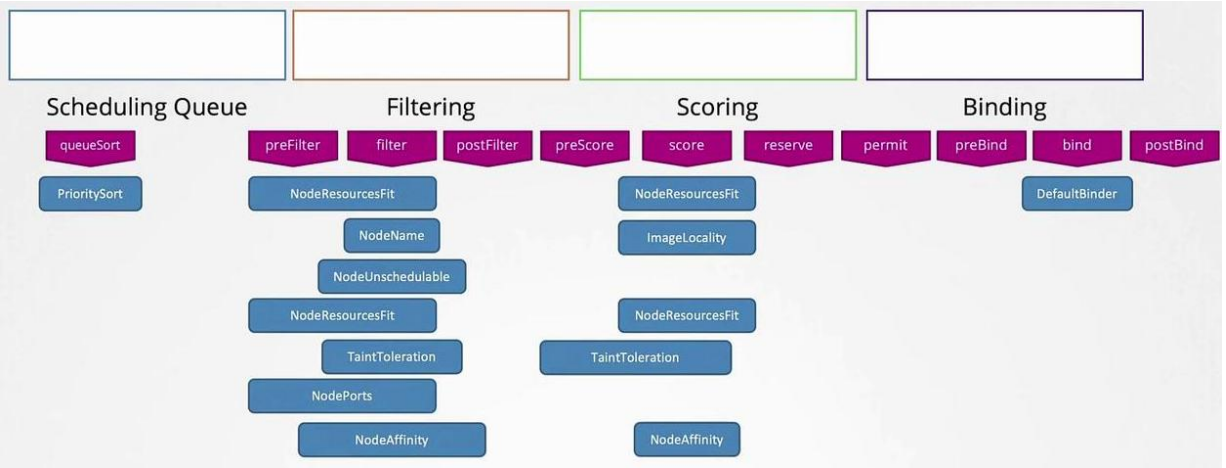
    Scheduler profile is a rule.
    - One scheduler can have multiple scheduler profiles.
  - Scheduling has different stages called ***execution point***.

    queueSort,

    preFilter, filter, postFilter,

    preScore, score, reserve,

    permit, preBind, bind, postBind
  - At each extension point, modules are bound.



  - Scheduler profiles runs on different threads.

    In each scheduler profile, you can decide which plugins has to be

enabled or disabled.

```yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler-2
  plugins:
    score:
      disabled:
      - name: TaintToleration
      enabled:
      - name: MyCustomPluginA
      - name: MyCustomPluginB
```

- KubeSchedulerConfiguration is not a Kubernetes object.
  That's why this file's path is given in kube-scheduler.yaml

- Leader Selection
  - *kubeschedulerconfiguration.leaderElect: true* (or) *false*
  - Enable: one scheduler will be active at a time
    Disable: multiple schedulers will be active at a time.
  - In case of race condition, Kubernetes handles it.
    Only one scheduling can happen. No re-scheduling.
  - Leader election should intelligently be chosen.
    scheduler -> s1: p1 p2 (profiles)
    scheduler -> s2: p3 p4
    if you have made *leader election* true, the pod has *schedulerName*: p3 and if s1 is chosen as leader, it doesn't have p3, pod will be in pending state forever.

- Types of *Authorizations mechanisms*:
  - Node Authorizer [for kubelet]
  - ABAC [Attribute Based Access Control]
  - RBAC [Role Based Access Control]
  - Webhook Mode

- Authentication in Kubernetes
  - Kubernetes itself doesn't provide authentication by itself because it doesn't have database to store credentials.

It depends on external providers like ***static file, certificate authorities, third party identity provider*** *like LDAP*.

- All authentication & authorization is done at API Server level, because it is the single point of contact to access the cluster.

- Static file authentication
  - csv file is maintained to write the password/token, username, user id, group id (optional)
  - `password123,user1,u0001,group1` (password)

    `KpjCvBI7rCFAHYPKByTlzRb7gulcUc4B,user10,u0010,group1` (token)

    [group is optional]
  - Now mention this file's path in the *kube-apiserver* running service (either running as a process or running inside a pod)
    - ```
      ExecStart=/usr/local/bin/kube-apiserver \
        --authorization-mode=Node,RBAC \
        --advertise-address=... \
        --basic-auth-file=user-details.csv
      ```
    - ```
      command:
        - kube-apiserver
        - --authorization-mode=Node,RBAC
        - --advertise-address=172.17.0.107
        - --basic-auth-file=/path/to/user-details.csv
      ```
  - Curl command usage:

    curl -v -k https://master-node-ip:6443

    for password: **-u "username:password"**

    for token: **--header "Authorization: Bearer 45urtgfhy9thgi5494"**

- Certificate generation process:
  - One entity (or user) will create a key (signature).

    it'll create a certificate about its identity and sign that with its signature [self-signed]. <---- **CA**
  - It'll be told that, that guy is the source of truth.

    If he signs in any certificate, then that certificate will valid (proof).
  - Now, any other user will create a *certificate signing request* and the certificate authority (CA) will sign this with its *key* (signature).

    now, that user will get a certificate which will be proof of it to be valid.

- In Kubernetes,

  one user will generate *key*, then a *csr*.

  csr will be self-signed, then *crt* will be generated.

  At the end: **ca.key, ca.crt** (signature, CA's identity proof)

  - For all the other users, they will create *key, csr*.

    crt will be generated with sign of *CA*.

- Client & Server components in Kubernetes
  - ONLY CLIENT

    kubectl, kube-scheduler, kube-controller-manager, kube-proxy

    [all talks to API Server]
  - BOTH SERVER and CLIENT

    kube-api-server

    [client: *etcd, kubelet* ; server: *kubelet,* other components]

    kubelet

    [client: *api-server* ; server: *api-server*]

    etcd

    [client: peer-to-peer ; server: *api-server*]
- **OpenSSL** commands
  - **openssl generate -out** <key file name>**.key 2048**

    create a key of size 2048 and store in a file.

    openssl genrsa -out server.key 2048
  - **openssl req -new -key** <key file name>**.key -subj "/CN=**<name>**/O=**<group>**" -out** <csr file name>**.csr**

    create certificate signing request file.

    Openssl req -new -key server.key -subj "/CN=etcd" -out server.csr
  - **openssl x509 -req -in** <csr file name>**.csr -CA** <cs crt file name>**.crt -CAkey** <ca key file name>**.key -out** <crt file name>**.crt**

    create a certificate signed by CA.

    openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -out server.crt
  - **openssl x509 -in** <crt file name>**.crt -noout -subject**

    display subject of the certificate (CN, O ..etc)
  - **openssl x509 -in** <crt file name>**.crt -noout -issuer**

    display the CA who signed this certificate.

- Encode and Decode ***base64*** format
  - To convert *key* or *crt* file into *base64* format,

    cat <crt file name>.crt | base64

    cat <crt file name>.crt | base64
  - To convert *base64* to *key* or *crt*

    cat <base64 content file name> | base64 -d
- **etcd peer-to-peer communication**
  - All etcd are *etcd servers*. ~~[no etcd client; nothing exist like that]~~
  - All *etcd servers* contains same data [data replication].
  - etcd servers end-points are *static*.
  - All the *etcd servers* end-points are provided to *kube-api-server*.

  - *kube-api-server* just connect to one *etcd server's end-point*,

    etcd cluster handles *leader selection, data consistency, request routing*.
- TLS in Kubernetes
  - Generate **CA** key and certificate.

    Key: **openssl genrsa -out ca.key 2048**

    CSR: **openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr**

    CRT: **openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt** [self signed]
    - CN: KUBERNETES-CA is just a *convention*, its never used logically.

      Trust is built with ***ca.crt*** itself.

      Keeping this name helps in analyzing logs and all.
  - Admin (kubectl) key and certificate (Client)

    Key: **openssl genrsa -out admin.key 2048**

    CSR: **openssl req -new -key admin.key -subj "/CN=kube-admin/O=system:masters" -out admin.csr**

    CRT: **openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt** [signed by CA]
    - /O : organization (or) group
    - /CN : common name (or) username

- **system:masters** group is bound with **cluster-admin** (cluster role).
  so, user having certificate containing group *system:masters* will have admin authorization.
- To find these:
  search for specific roles or cluster roles
  search for rolebindings or clusterrolebindings binding those role/clusterrole

- Similarly, generate key and certificates for other clients (CN will only be changed).
- Server keys and certificates will be created same way.
- etcd key and certificate
  same command, CN=etcd
  - **etcd has its own CA** (not same as Kubernetes CA)
    it doesn't talk to any other component as a client. Only talks to peer etcd servers.
  - Its CA has name: **etcd-ca**
  - *etcd* has 2 certificates: *server.crt, peer.crt* [both are signed by *etcd CA*]
  - kube-api-server connectes to etcd: *server.crt*
    etcd connects to peer etcd: *peer.txt*
    ```
    - command:
      - etcd
      - --cert-file=/etc/kubernetes/pki/etcd/server.crt

      - --peer-cert-file=/etc/kubernetes/pki/etcd/peer.crt
      - --peer-client-cert-auth=true
      - --peer-key-file=/etc/kubernetes/pki/etcd/peer.key
      - --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt

      - --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
    ```
  - Kubernetes CA & etcd CA don't trust each other.
    They both are completely independent.
- kube-api-server key and certificate (Server & Client)
  - 2 certificates, in both CA are different.
  - One signed by Kubernetes's CA
    another signed by etcd CA

- *certificate* is not global. It is used per connection.
- kube-api-server as server: Kubernetes's CA signed crt
  kube-api-server as client: etcd CA signed crt.
- apiserver-etcd-client.crt : talking as a client to etcd
  apiserver-kubelet-client.crt : talking as a client to kubelet

- kubelet key and certificate (Server & Client)
  - Server certificates: (api server as client)
    client certificate (talking to other peers)
  - For client certificate, the naming format must be
    system:node:<*node name****>*** (username)
    system:nodes (group)
    system:node:node01
    both username and group must be in this format. Otherwise not authorized. (Node Authorizer)

- SSL verification flow
  - Client sends request
  - Server asks for certificate
  - Client sends certificate
  - Server verfies CA.
    Extracts name (CN) & group (O) then applies RBAC.

- **kubeconfig**
  - Config file contains 3 main things: clusters, contexts, users
  - **clusters**
    it a list. Each index contains 1 cluster's details, i.e. *name* (just an identifier), *cluster's end-point* (cluster's api server's end point [6443 port]), *cluster's CA certificate*.
  - **users**
    its also a list. Each index conains 1 user's details, i.e. *name* (just an identifier), *client key* and *client certificate*.
  - **contexts**
    its also a list. Each index contains a context.
    context = user + cluster + *default* namespace.
    each context contains a *name* (just an identifier), *user* (name of the

user), *cluster* (name of the cluster), *namespace* (optional; if not given *default* namespace will be considered by default)

- when you execute any command, by default it'll append
  --namespace=<provided namespace in context**>**
  however, you can override this by giving
  --namespace=<some other namespace**>**
- Context name convention: *<user name>@<cluster name>*

- **current-context** : shows the context you are currently in.
- ~~*User or cluster can't be switched*~~*, need to switch context to get a particular user & cluster pair.*
- **kubectl config view**
  displays the config file.
  kubectl config view --raw
  it'll display certificates and keys as well.
- **kubectl config use-context <**context name**>**
  to switch context (updates *current-context* field in config file)
- **kubectl config set-context <**context name**> --cluster=<**cluster name**> --user=<**user name**> --namespace=<**namespace name**>**
  if context exists: update that; if not: create a new context.
  cluster name & user name should be there in config file.
  namespace is optional.
- Default kube config file path: **~/.kube/config**
  to use any other config file append
  --kubeconfig <config file path**>**
  kubectl get pods --kubeconfig /path/to/config

- Use **jsonpath** to extract value of specific key.
  kubectl config view --raw -o jsonpath='{.users[0].user.client-key-data}'
- To communicate with api server with directly http REST call
  curl https://<controlplane node ip**>:6443 --cert** /path/to/crt **--key** /path/to/key **--cacert** /path/to/ca.crt
  [only in case of certificate authentication, otherwise authorization or username password]

- GET request to api-server end-point gives all the available REST paths

```
"/apis/",
"/apis/admissionregistration.k8s.io",
"/apis/admissionregistration.k8s.io/v1",
"/apis/apiextensions.k8s.io",
"/apis/apiextensions.k8s.io/v1"
```

- Kubernetes Resource API
  - **/api** and **/apis** contains all the resources.

    Other paths like /metrics, /healthz, /version, /logs ..etc are operational, debugging or meta end-points.
  - **/api** : core Api root path (legacy)

    /apis : named Api root path (modern)
  - 3 things are there to be precised:

    resources (pod, events, replicaset ..etc)

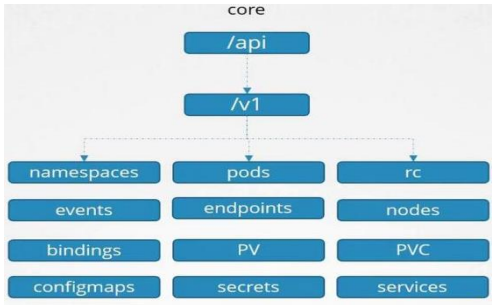
    resources versions (currently all are v1 only)

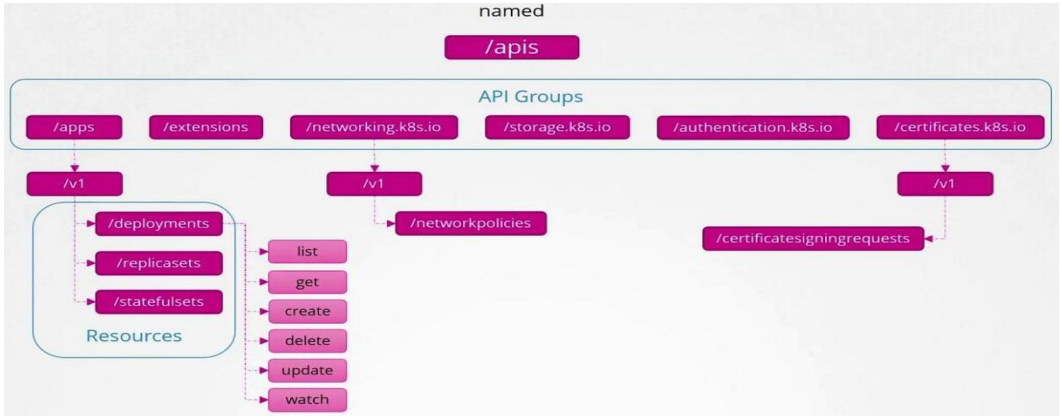
    resources types (resources are grouped; called API Groups)
  - resource ⊂ version ⊂ API groups (⊂ : subset)
  - **/api** is legacy, it doesn't contain api groups.

    It contains version directly (**v1**) and resources inside those.
  - **/apis** is modern, it contains api groups.

    Each api group contains version (currently **v1** only).

    each version contains resources.





- Each resource has **verbs** (get, list, create, delete ..etc)

  actions that can be performed with resources.
- In *yaml* file, against *apiVersion* field:

  "<version>" : (/api), because api group is not present here

  "<api group>/<version>" : (/apis), api groups are here

  apiVersion: "v1"

  apiVersion: "apps/v1"
- official docs

- Authorization mechanisms are provided in *kube-apiserver* command (for pod) or ExecStart (for process)

```
- command:
  - kube-apiserver
  - --authorization-mode=Node,RBAC
```

  - **--authorization-mode=<**mechanism 1**>,<**mechanism 2**>,** ..etc
  - It is executed from left to right until the client is allowed.

    If till the last authorization mechanism all rejects, then client is unauthorized.

- **ServiceAccount**
  - Unlike User account, ServiceAccount is used for services (internal components or external tools like jenkins, prometheus) to communicate with API Server.
  - Example: the pods contains *default ServiceAccount* by default.

```
vagrant@controlplane:~/.kube$ kubectl describe pods dev-pod
Name:             dev-pod
Namespace:        default
Priority:         0
Service Account:  default
```

    [~~inter pod communication doesn't require API server~~]

    [multiple reasons for Pod to be needing ServiceAccount to talk to API server]
  - ServiceAccount is a Kubernetes resource (core API root path)

    [long lived object]
  - For internal components (like pods ..etc)

    token is created linked to the ServiceAccount by default.

For external tools

token has to be created manually.

- token is required to communicate with API server.

- **kubectl create token** <service account name>

  [token is not a long lived resource]

  [token linked to a ServiceAccount has all the authorization provided to ServiceAccount]

- a virtual volume is created and mounted to the pod which contains ServiceAccount related things.

```
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-rdvhr (ro)
```

  virtual volume lifecycle = pod's life cycle [if pod is gone, virtual volume is gone]

  to disable this auto mounting of volume:

  pod.spec.automountServiceAccountToken: false

- **Bearer** <token> can be used in headers to communicate.

- Node Authorizer (Authorization mechanism 1)
  - Used by ***kubelet***. Its certificate contains:

    system:node:<node name> [name (CN)]

    system:nodes [group (O)]
  - This convention cannot be changed. Its fixed for Kubernetes Node Authorizer.

- **ABAC** [Attribute Based Access Control] (Authorization mechanism 2)
  - It uses a policy file (JSON file).

    contains list of dictionaries (each dictionary: policy)

```
{
  "user": "viewer",
  "apiGroup": "*",
  "resource": "*",
  "namespace": "*",
  "readonly": true
},
```

    ["*" means all access]

    [**""** in apiGroup doesn't mean all access, it means core api root path]

-
```
{
  "user": "monitor",
  "nonResourcePath": "/metrics",
  "readonly": true
}
```
[for non resource paths]
- Policy file path is given in *kube-apiserver* command:
```
--authorization-mode=ABAC
--authorization-policy-file=/etc/kubernetes/abac-policy.json
```
- ~~Not Preferable; large policy file; complex to handle;~~

- **RBAC** [Role Based Access Control] (Authorization mechanism 3)
  - It consists of 4 main things:

    Role, RoleBinding, ClusterRole, ClusterRoleBinding

    [comes under '*rbac.authorization.k8s.io*' api group]
  - Role
    - **rules** list will be here.
    - Each rule contains: apiGroup (list), resources (list), verbs (list)
    -
```
rules:
- apiGroups: [""] # "" indicates the core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```
    - Role is namespace scoped.
```
metadata:
  namespace: default
  name: pod-reader
```
    - If namespace field is not provided,

      it'll be created on the namespace present in ***current-context***

      (or)

      --namespace <namespace name> in this namespace
    - Within resource (ex: pod), if don't want to give access to all resources (ex: all pods)

      resourceNames is the key.
```
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```
      [only pods with names "blue" & "orange" will be accessible]

- ClusterRole
  - Same as Role, but it is not namespace scoped.

    It is **cluster-scoped**.
  - No need to provide *namespace* field.
- RoleBinding
  - Maps subject to roles (Role, ClusterRole).

    [subject: certificate subjects, CN: user, O: group ...etc]
  - It is **namespace-scoped**.
  - ```yaml
    subjects:
    - kind: User
      name: dev-user
      apiGroup: rbac.authorization.k8s.io
    roleRef:
      kind: Role
      name: developer
      apiGroup: rbac.authorization.k8s.io
    ```
    - here **kind** is not same as top level kind i.e. *kind: Pods*
    - kind in roleRef: *Role* or *ClusterRole*
    - kind in subjects: *User, Group,* or *ServiceAccount*
    - apiGroup in *subjects* seems unnecessary (User, Group)

      ServiceAccount is a k8s resource, present in core api root path (*apiGroup: “”*).
    - Internally (kind + apiGroup) matching is there, so apiGroup is needed.
  - Here also, **namespace** should be given;

    if namespace given: *RoleBinding* object will be created in that.

    If not, *RoleBinding* object will be created in *current-context* or --namespace (if provided) namespace.
  - Role provided in RoleBinding must be present in same namespace.

    If ClusterRole, then no worries.
  - Only Resources in same namespace where RoleBinding exist will be authorized whether the role is Role or ClusterRole.
- ClusterRoleBindnig
  - It is cluster-scoped.
  - It can only map to ClusterRole. ~~Cannot map Role~~.
  - Resources through out the cluster will be authorized in here.
    ```

- After creating binding (RoleBinding or ClusterRoleBinding), ~~you cannot change *roleRef* in imperative way~~.
- Roles (Role, ClusterRole): which resources and actions can be accessed.
  RoleBinding (RoleBinding, ClusterRoleBinding): maps subjects to roles
  [subject: certificate subjects, CN: user, O: group ...etc]

- Webhook (Authorization mechanism 4)
  - Uses third-party REST service to get the authorization about a user or service account.
  - Ex: Open Policy Client

- **AlwaysAllow** and **AlwaysDeny** (Authorization mechanism 5 & 6)
  - When no authorization mechanism provided in **--authorization-mode** *AlwaysAllow* is the default one.
  - **AlwaysAllow** allows everyone to access everyone.
    AlwaysDeny restricts everyone's access to everything.

- To see if a user is authorized (**auth can-i**):
  kubectl auth can-i create deployments
  yes or *no* (output)
  To see if another user is authorized or not (**--as** <user name>)
  kubectl auth can-i create deployments --as dev-user
  yes or *no* (output)

- Image Security
  - Default docker registry: *docker hub*
  - ***image*:** nginx
    registry: docker.io
    namespace (organization/user): library (official images present in this)
    repository: nginx
    tag: latest
  - *image*: nginx ==> *image*: docker.io/library/nginx
  - *image*: private-registry.io/apps/internal-app
    [if you want to specify other registry or other namespace]
  - In case of private registry, you need to authenticate in that.
  - Create a **Secret** (Kubernetes Resource)
    mention secret object's name in *pod.spec.**imagePullSecrets.name*** field.

- **kubectl create secret docker-registry** <secret name> \
  --docker-server=<docker registry server> \
  --docker-username=<docker registry username> \
  --docker-password=<docker registry password> \
  --docker-email=<docker registry email>
  - docker-registry => Create a secret for use with a Docker registry
- ```
  spec:
    imagePullSecrets:
    - name: my-registry-secret-1
    - name: my-registry-secret-2
  ```
  *pod.spec.**imagePullSecrets*** is a list; it'll try one by one until it finds the required image.
  [containers and imagePullSecrets: both are siblings]
- Create **secret** from yaml

  ```
  apiVersion: v1
  data:
    .dockerconfigjson: eyJhdXRocyI6eyJ
  kind: Secret
  metadata:
    name: my-secret
  type: kubernetes.io/dockerconfigjson
  ```
  [yaml file looks something like this]
  - *secret.data.\.dockerconfigjson* (**\.** means to keep **.** as a string)
  - That is **base64** encoded.
    On decoding that:

    ```
    {
      "auths": {
        "DOCKER_REGISTRY_SERVER": {
          "username": "DOCKER_USER",
          "password": "DOCKER_PASSWORD",
          "email": "DOCKER_EMAIL",
          "auth": "RE9DS0VSX1VTRVI6RE9DS0VSX1BBU1NXT1JE"
        }
      }
    }
    ```
    [something like this will come]
- official docs

- Security Context
  - Docker containers default user: **root** [root's user ID: 0]
    even *root* user doesn't have all capabilities. You need to specifically add.
  - **docker run --user=**<user id> <image name> [user ID will be given]
    docker run --cap-add <capability name> <image name> [add capability]
    ```

- `NET_ADMIN` → control networking
  `SYS_ADMIN` → system-level control
  `CHOWN` → change file ownership
  `MAC_ADMIN` → control security policies
- In kubernetes, either you can define these in ***pod level*** or ***container level***.
  Pod level: *pod.spec.**securityContext*** [write the configuration here]
  Container level: *pod.spec.containers[*].**securityContext***
- ```
  securityContext:
    runAsUser: 1000
    capabilities:
      add: ["MAC_ADMIN"]
  ```

- **NetworkPolicy** (*apiVersion*: *networking.k8s.io/v1*)
  - It is **namespace scoped**.
  - Default behaviour: any pod can communicate with any other pod.
    Network Policies: restrict those behaviour.
  - Example: only backend should talk to database; not frontend;
  - Ingress: incoming traffic
    egress: outgoing traffic
  - Traffic restriction will work only to the pods where NetworkPolicy is applied.
    To link pods, ***selector*** and ***labels*** method only is used here as well.
  - *networkpolicy.spec.**podSelector***
    select the pod on which NetworkPolicy has to be applied i.e. on which ingress and egress traffic restriction will be applied.
  - *networkpolicy.spec.**ingress***
    rules for allowed ingress traffic to the pod. (target pod's selector, ports)
  - *networkpolicy.spec.**egress***
    rules for allowed egress traffic
  - *networkpolicy.spec.**policyTypes***
    it enforces the ingress and egress rules.
    - If *policyTypes* field is not there, Kubernetes will add the values (Ingress, Egress) based on presence of *networkpolicy.spec.ingress* and *networkpolicy.spec.egress* fields.
      If only ingress is present, it becomes Ingress; if only egress is present, it becomes Egress; if both are present, it becomes both
  ```

Ingress and Egress.

In the live configuration (**kubectl get networkpolicy** <name> **-o yaml**), the *policyTypes* field will always be present, either defined by the user or added by Kubernetes.

- In the live config, whatever values are there [Ingress or Egress or both] inside *networkpolicy.spec.policyTypes*, only those will be enforced.

  If a direction [Ingress or Egress] is listed in policyTypes but no corresponding rules are defined, then that direction becomes completely blocked (**default deny**).

  For example, if policyTypes contains Ingress but no ingress rules are defined, then all incoming traffic is blocked.

```yaml
apiVersion: networking.k8s.io/
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 172.17.0.0/16
        except:
        - 172.17.1.0/24
    - namespaceSelector:
        matchLabels:
          project: myproject
    - podSelector:
        matchLabels:
          role: frontend
    ports:
    - protocol: TCP
      port: 6379
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 5978
```

- *networkpolicy.spec.ingress.from.**namespaceSelector***

  if provided, namespaces with those labels will be selected.

If not provided, pods within the same namespace (~~not all namespaces~~) will be allowed.

々 F

々 F

々 F

々

々　F

々　F

々　F

々　F

々　F

々